

LTU-RAI Assignment

Rushikesh Patil

August 2026

The following github link contain all the source codes used in the task. [LTU-RAI-Assignment](#)

1 Introduction

Recent advances in autonomous robotics have enabled teams of robots to operate cooperatively in complex environments. Multi-robot systems offer significant advantages over single-robot solutions, including increased spatial coverage, improved robustness to failures, parallel task execution, and faster mission completion. These capabilities are particularly important in applications such as industrial inspection, search and rescue, environmental monitoring, warehouse automation, and autonomous exploration.

A fundamental challenge in multi-robot autonomy is the development of mission planning frameworks that can efficiently allocate tasks among multiple agents while maintaining safety and coordination. In addition, modern robotic systems increasingly rely on semantic understanding of the environment and natural language interaction, requiring the integration of perception, planning, and reasoning capabilities. Recent developments in foundation models and large language models (LLMs) provide new opportunities for translating human instructions into executable robotic plans. However, ensuring that such plans remain grounded, safe, and adaptable to dynamic environments remains an open research challenge.

This work addresses these challenges through two complementary components. First, a centralized multi-robot mission planning framework is developed using ArduPilot, ROS2 Humble, Gazebo Garden, and RTAB-Map. Multiple aerial robots are assigned a set of mission waypoints while ensuring that no waypoint is visited by more than one robot. The system further supports dynamic task reassignment in the event of robot failures, enabling continued mission execution.

Second, a semantic mission planning framework is proposed in which natural language instructions are converted into executable task plans using a foundation model. A semantic scene graph is used to represent objects and their attributes within the environment. The generated plans are subsequently grounded to real scene entities, validated through a safety verification layer, and modified through an event-driven replanning mechanism when environmental changes occur.

The contributions of this work are summarized as follows:

- Development of a centralized multi-robot mission planner capable of waypoint allocation and failure recovery.
- Integration of RGB-D based localization and mapping using RTAB-Map within a multi-robot simulation environment.
- Design of a semantic planning architecture consisting of scene graph generation, grounding, validation, and replanning modules.

- Demonstration of event-driven replanning and safety validation for natural-language mission execution.
- Discussion of long-term deployment challenges and the role of foundation models in future autonomous robotic systems.

The remainder of this report is organized as follows. Section 2 presents the multi-robot simulation environment and centralized mission planning framework. Section 3 describes the semantic mission planning architecture, including grounding, validation, and replanning. Section 4 discusses challenges associated with long-term robot deployment and the safe integration of foundation models. Finally, Section 5 concludes the report and outlines future research directions.

2 Multi-Robot Gazebo Simulation Environment

A realistic multi-robot simulation environment was developed in Gazebo Garden to evaluate autonomous mission execution and coordination between multiple aerial robots. The environment was designed to resemble a small industrial and inspection scenario containing both static infrastructure and mobile assets. The simulated world includes buildings, parked vehicles, trucks, open terrain, and a runway-like operating area that provides sufficient space for autonomous navigation and waypoint-based missions.

The environment contains multiple structures representing warehouses and industrial buildings. These structures serve as visually distinctive landmarks for localization and semantic reasoning tasks. Several vehicles and trucks are distributed throughout the scene to introduce additional environmental complexity and provide objects that may be referenced during semantic mission planning. The inclusion of multiple object categories enables the construction of a semantic scene graph in later stages of the assignment.

Two autonomous aerial robots, denoted as r_0 and r_1 , are deployed within the simulation. Each robot is equipped with an RGB-D camera and an inertial measurement unit (IMU). The RGB-D camera provides synchronized color and depth information that is used for localization and mapping, while the IMU provides orientation and motion measurements required for state estimation and flight control.

The robots operate using ArduPilot SITL and communicate through MAVLink interfaces. A ROS2 Humble middleware layer is used to integrate sensor streams, state estimation modules, and mission planning components. RTAB-Map is employed for RGB-D odometry and localization, allowing each robot to estimate its pose within the simulated environment. The resulting pose estimates are used by the centralized mission planner for waypoint navigation and task allocation.

Figure 1 illustrates the simulation environment used throughout this work. The world contains multiple inspection targets, buildings, vehicles, and open operational areas that support autonomous exploration and mission execution.

The primary objective of the simulation environment is to provide a controlled yet sufficiently complex setting for evaluating multi-robot coordination strategies. The environment supports the execution of waypoint-based missions, dynamic task reassignment, semantic reasoning experiments, and event-driven replanning scenarios. By combining realistic sensing, localization, and autonomous control, the simulation serves as a representative testbed for the proposed multi-robot mission planning framework.

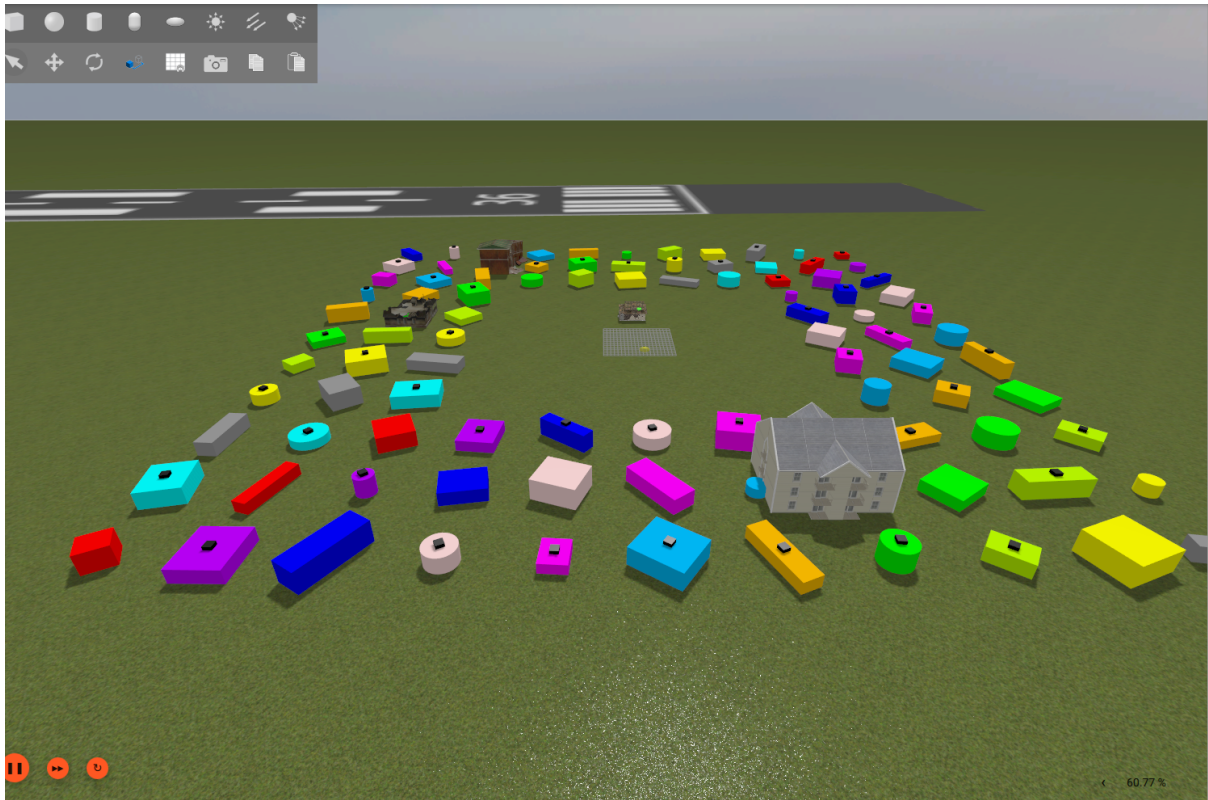


Figure 1: Gazebo simulation environment used for multi-robot mission planning experiments. The environment contains buildings, trucks, vehicles, a runway area, and two autonomous aerial robots.

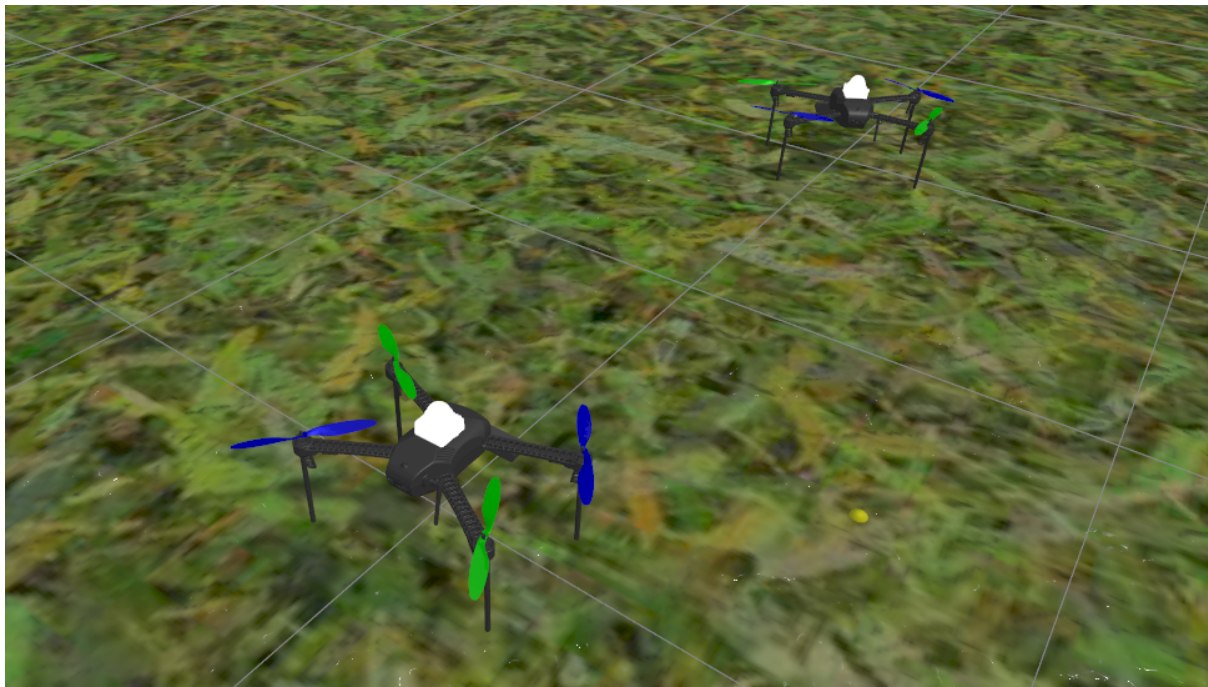


Figure 2: Drone in Gazebo environment

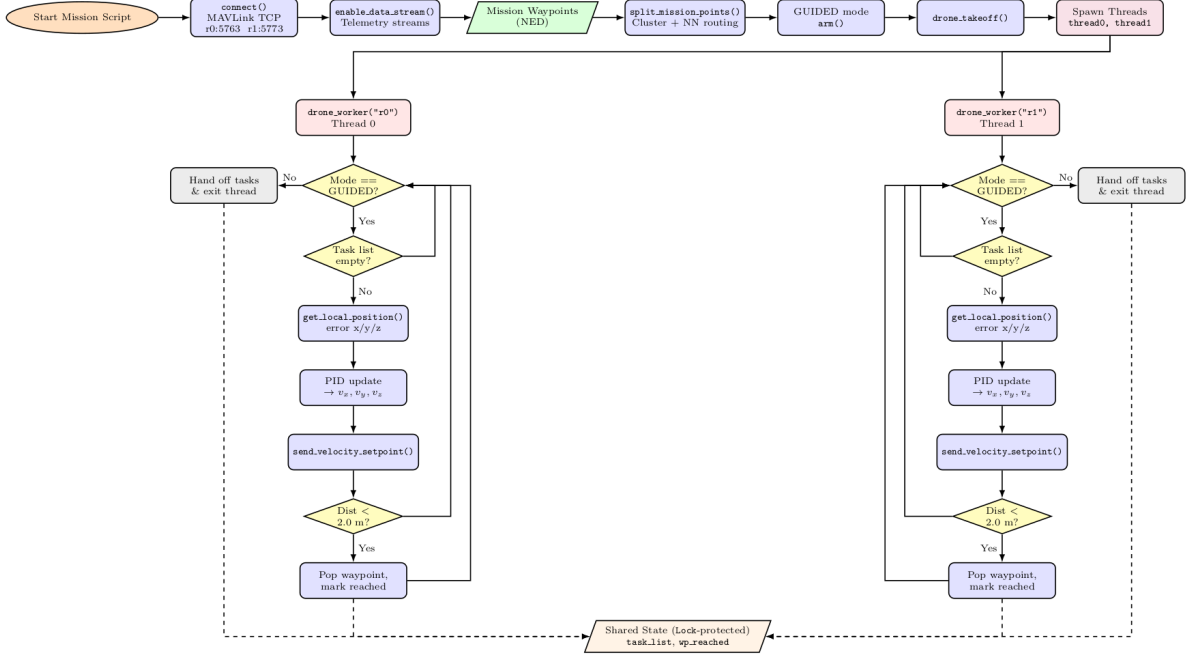


Figure 3: Mission Control Flow

2.1 Centralized Mission Planning

A centralized mission planner was developed to coordinate two aerial robots during waypoint-based exploration missions. The planner maintains a global list of mission waypoints and assigns each waypoint to exactly one robot, ensuring that no duplication of effort occurs during mission execution [1].

2.2 Waypoint Generation

A set of $N = 10$ three-dimensional mission waypoints is generated within the operational area. Each waypoint is represented by

$$W_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}, \quad i = 1, \dots, N \quad (1)$$

where x_i , y_i , and z_i denote the waypoint coordinates in the local NED reference frame.

The complete mission set is

$$\mathcal{W} = \{W_1, W_2, \dots, W_N\}. \quad (2)$$

2.3 Spatial Clustering and Route Assignment

The generated waypoints are partitioned according to their spatial proximity to the available robots. This strategy is inspired by clustering-based Multi-Robot Task Allocation (MRTA) approaches, where geographically similar tasks are grouped together before assignment in order to reduce travel distance and computational complexity [6] [2].

For each waypoint W_i , the Euclidean distance to every robot is computed and the waypoint is assigned to the nearest robot.

Let the initial positions of robots r_0 and r_1 be

$$P_0 = [x_0 \ y_0 \ z_0]^T, \quad (3)$$

$$P_1 = [x_1 \ y_1 \ z_1]^T. \quad (4)$$

For every waypoint W_i , the Euclidean distance to each robot is computed as

$$d_{0,i} = \|W_i - P_0\|_2, \quad (5)$$

$$d_{1,i} = \|W_i - P_1\|_2. \quad (6)$$

The waypoint is assigned to the robot with the minimum distance,

$$A(W_i) = \arg \min_{r \in \{r_0, r_1\}} d_{r,i}. \quad (7)$$

This produces two mutually exclusive waypoint sets,

$$\mathcal{W}_{r_0} \cap \mathcal{W}_{r_1} = \emptyset. \quad (8)$$

Therefore, no waypoint can be visited by more than one robot.

2.4 Waypoint Tracking

Waypoint tracking architectures have been widely used for autonomous UAV navigation and guidance [4, 7].

For a target waypoint

$$W_t = [x_t \ y_t \ z_t]^T \quad (9)$$

and current robot position

$$P = [x \ y \ z]^T, \quad (10)$$

the tracking error is

$$e_x = x_t - x, \quad (11)$$

$$e_y = y_t - y, \quad (12)$$

$$e_z = z_t - z. \quad (13)$$

The three-dimensional position error vector is

$$\mathbf{e} = [e_x \ e_y \ e_z]^T. \quad (14)$$

2.5 PID Velocity Controller

Velocity commands are generated using independent PID controllers for the North, East and Down directions. PID control remains one of the most widely used approaches for trajectory and waypoint tracking due to its simplicity, robustness and ease of implementation [3].

The proportional term is

$$P(t) = K_p e(t). \quad (15)$$

The integral term is

$$I(t) = K_i \int_0^t e(\tau) d\tau. \quad (16)$$

The derivative term is

$$D(t) = K_d \frac{de(t)}{dt}. \quad (17)$$

The resulting velocity command is

$$v(t) = P(t) + I(t) + D(t). \quad (18)$$

The commanded body-frame velocity vector is

$$\mathbf{v} = [v_x \quad v_y \quad v_z]^T. \quad (19)$$

2.6 Waypoint Completion

A waypoint is considered reached when the Euclidean distance between the robot and the target waypoint satisfies

$$\|W_t - P\|_2 < d_{th} \quad (20)$$

where

$$d_{th} = 2.0 \text{ m}. \quad (21)$$

The waypoint is then marked as completed and removed from the robot's task queue.

2.7 Failure Recovery

Mission robustness is improved through dynamic task reassignment. Let

$$\mathcal{R} = \{r_0, r_1\} \quad (22)$$

represent the set of active robots.

If robot r_i leaves GUIDED mode or becomes unavailable,

$$r_i \notin \mathcal{R}, \quad (23)$$

all remaining waypoints assigned to that robot,

$$\mathcal{W}_{r_i}^{\text{remaining}}, \quad (24)$$

are transferred to the remaining active robot

$$\mathcal{W}_{r_j} = \mathcal{W}_{r_j} \cup \mathcal{W}_{r_i}^{\text{remaining}}. \quad (25)$$

This allows mission execution to continue despite robot failures.

2.8 Failure Scenarios

Failures are inevitable in the real world and that has to be accounted for while deploying a system. In the given algorithm if N robots are there and X no of robots fail during the mission then the remaining waypoints would be split into $(N-X)$ drones ensuring that no two drones visit the same waypoint. in the given simulation failure was simulated by giving RTL/LAND command because these command are normally used for deliberate or emergency actions.

2.9 Results

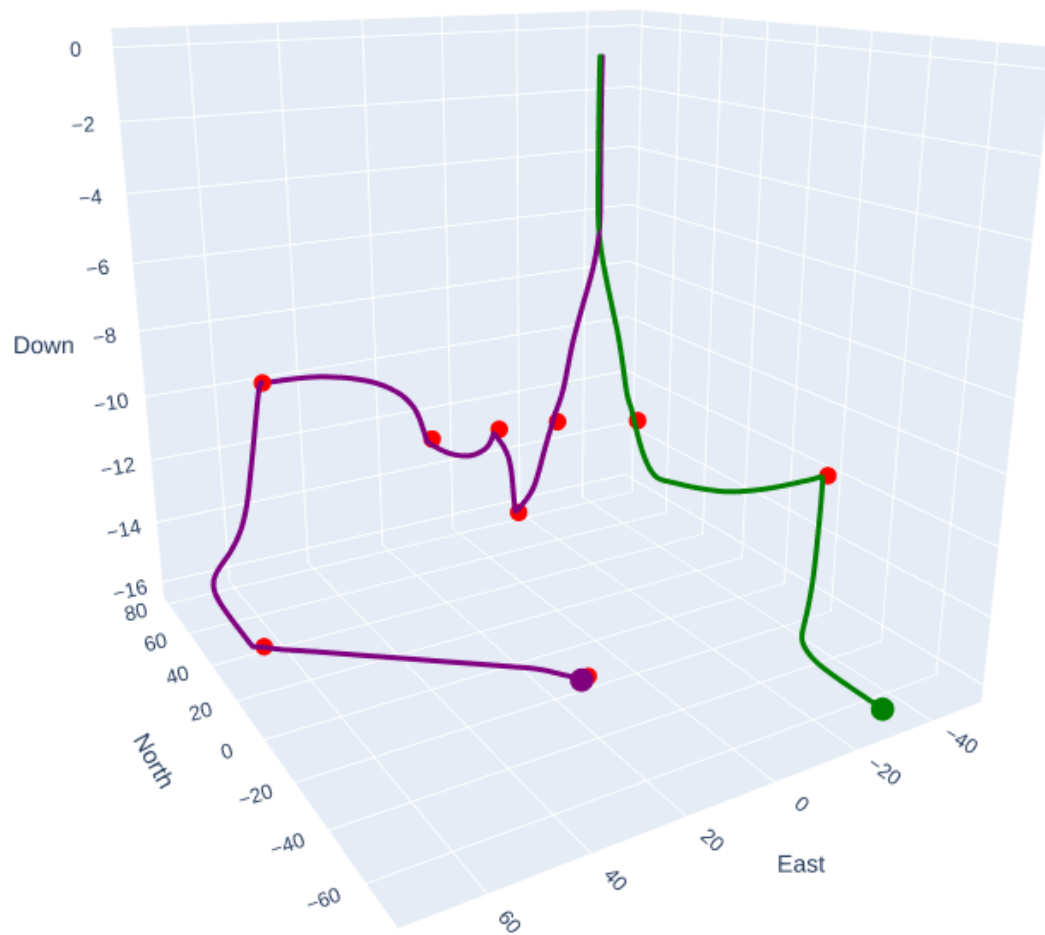


Figure 4: Trajectory (NED) Plot of r_0 , r_1 of mission (Red Dots: Mission Waypoints, Green Path: r_0 , Purple Path: r_1)

[Interactive 3D Plot](#)

Table 1: Waypoint Allocation Generated by the Spatial Clustering Algorithm

Robot	Assigned Waypoints (North, East, Down)
r0	(0, -10, -10)
	(-20, -50, -12)
	(-70, -30, -16)
r1	(10, 5, -10)
	(-35, 20, -15)
	(80, 50, -10)
	(20, 10, -13)
	(60, 15, -12)
	(35, 8, -11)
	(18, 70, -15)

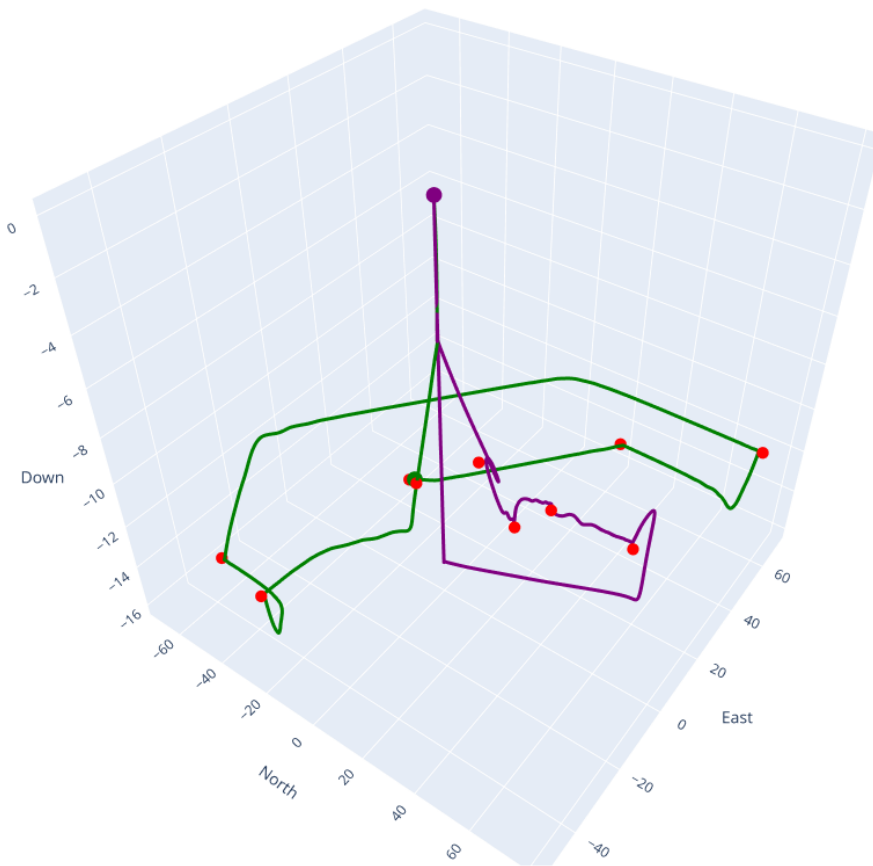


Figure 5: Improvised Trajectory (NED) of r0 when r1 fails (Red Dots: Mission Waypoints, Green Path: r0, Purple Path: r1)

[Interactive 3D Plot](#)

Table 2: Waypoint Allocation During Failure Recovery

Robot	Assigned Waypoints (North, East, Down)
r_0	(0, -10, -10)
	(-20, -50, -12)
	(-35, 20, -15)
	(80, 50, -10)
	(-70, -30, -16)
	(18, 70, -15)
r_1	(10, 5, -10)
	(20, 10, -13)
	(60, 15, -12)
	(35, 8, -11)

Figure 4 shows the nominal mission execution scenario. Ten randomly generated waypoints were distributed between two UAVs using the centralized mission planner. The assigned trajectories are shown in green for Robot r_0 and purple for Robot r_1 , while red markers represent mission waypoints. Both robots successfully navigated to their assigned waypoints using the PID velocity controller and completed the mission without conflicts. Since waypoint allocation enforces a mutually exclusive assignment strategy, no waypoint was visited by more than one robot.

Figure 5 illustrates a fault-tolerant mission execution scenario. During the mission, Robot r_1 experienced a simulated failure event and was commanded to execute a Return-To-Launch (RTL) procedure. The centralized planner detected the loss of the robot and transferred all unfinished tasks to Robot r_0 . As a result, Robot r_0 continued the mission and completed the remaining waypoints that had originally been assigned to Robot r_1 . This experiment demonstrates dynamic task reassignment and mission continuity under robot failure conditions.

The results confirm that the proposed centralized mission planner satisfies the task requirements by ensuring unique waypoint visitation, successful mission completion under nominal conditions, and adaptive reallocation of tasks during robot failures.

2.10 Limitations

The current implementation assumes reliable localization and does not explicitly model communication failures or sensor degradation. The waypoint allocation strategy is based on spatial clustering and does not optimize mission completion time globally. Furthermore, semantic planning is demonstrated in a simulated environment with a predefined scene graph and has not yet been validated on physical robotic platforms.

2.11 Future Work

Future work will investigate distributed mission planning, communication-aware task allocation, energy-aware route optimization, and integration of dynamic semantic scene graphs generated directly from onboard perception systems. Additionally, foundation models will be evaluated for autonomous mission generation in large-scale environments while maintaining safety through formal validation mechanisms.

3 Semantic Mission Planning and Agentic Task Decomposition with Foundation Models

3.1 Semantic Scene Graph

The semantic scene graph serves as the world model used by the foundation-model-based mission planner. The environment is represented as a collection of semantic nodes, where each node corresponds to a physical object that exists in the simulated environment. Examples include buildings, landing zones, vehicles, and emergency areas. Each node stores both semantic and metric information required for mission execution.

A node is represented as

$$S_i = \{id, type, position, attributes\} \quad (26)$$

where id uniquely identifies the object, $type$ defines its semantic category (e.g., building, vehicle, landing zone), $position = [x, y, z]$ specifies its location in the environment, and $attributes$ contains additional information such as accessibility, visibility, or operational status.

The complete scene graph is defined as

$$\mathcal{G} = \{S_1, S_2, \dots, S_n\} \quad (27)$$

where n denotes the number of semantic entities present in the environment.

For this assignment, the scene graph contains objects such as `building_a`, `building_b`, `landing_pad`, `vehicle_1`, and `emergency_zone`. The graph provides a structured representation of the environment that enables language grounding, plan validation, and event-driven replanning. Similar semantic world representations have been widely adopted in robot task planning and embodied AI systems because they provide an explicit mapping between language concepts and physical entities in the environment [8, 9].

3.2 Planner

The planner is responsible for converting a natural language mission into a structured sequence of executable actions. In the proposed framework, an open-source large language model (Llama 3.1 8B running through Ollama) is used as the reasoning engine. The mission description is provided as a prompt together with the available objects contained in the semantic scene graph. The model generates a task plan represented as a JSON action list, where each action contains an operation and a target object. For example, the mission “*Inspect Building A and then land on the landing pad*” is decomposed into navigation, inspection, and landing actions. This approach follows recent work demonstrating that large language models can perform high-level robotic task planning directly from natural language instructions [8].

3.3 Grounding

The grounding module takes the high-level plan generated by the language model and resolves every referenced object to a valid entity contained within the semantic scene graph. For each action in the plan, the target object is searched within the scene graph and its semantic and metric attributes are retrieved. If the requested object does not exist, the grounding process raises an exception and prevents further execution. Otherwise, the planner enriches each action with the object’s type and three-dimensional position, producing a grounded plan that can be executed by a robot. This step establishes a direct mapping between language and the physical environment, ensuring that all generated actions refer only

to objects that actually exist in the world representation. The grounded plan therefore serves as the interface between semantic reasoning and robot execution[8].

3.4 Validation and Execution

Before execution, every grounded action is passed through a validation layer to ensure that the generated plan is both feasible and safe. The validator performs several checks, including verifying that the target object exists in the scene graph, confirming that the object is currently visible, ensuring that landing actions are only performed on designated landing zones, and restricting inspection actions to valid building entities. Any violation results in immediate rejection of the plan and prevents unsafe execution. For example, a mission instructing the robot to land on a vehicle is detected and blocked before execution. Once validation succeeds, the plan is executed sequentially by the execution module, which processes each action in order. This validation-before-execution architecture is commonly used in autonomous robotic systems to ensure that high-level plans generated by AI models satisfy environmental and operational constraints before being acted upon [9]. Validation checks implemented in the framework are shown in [9] and include object existence, visibility, landing validity, and inspection validity.

3.5 Event-Driven Replanning

Real-world environments are dynamic and may change after a mission has been generated. To address this challenge, the framework includes an event-driven replanning module that monitors the execution process and reacts to changes in the environment. When a significant event is detected, such as target occlusion, object destruction, or loss of a valid landing location, the current plan is modified using the updated scene graph. In the presented implementation, if Building A becomes occluded, the mission is automatically redirected to Building B. Similarly, if the inspection target becomes unavailable, a new mission is generated to land at an emergency zone. The replanner therefore enables adaptive mission execution while maintaining mission objectives and safety constraints. Similar adaptive planning approaches have been explored for autonomous robots operating in dynamic environments [10].

3.6 Results

The following example demonstrates the complete semantic planning pipeline, including task decomposition, grounding, validation, execution, event-driven replanning, and failure recovery. The first scenario shows successful mission execution and replanning after target occlusion, while the second scenario demonstrates validation failure and generation of a safe alternative mission.

Table 3: Example execution of the semantic mission planning framework.

Stage	Output
Mission Planner	Inspect Building A and then land on the landing pad goto(Building A) → inspect(Building A) → goto(Landing Pad) → land(Landing Pad)
Grounding	Building A → (20,10,0), Landing Pad → (0,0,0)
Validation	Passed
Execution	Mission executed successfully
Event	Building A becomes occluded
Replanner	Replace Building A with Building B
Updated Mission	goto(Building B) → inspect(Building B) → goto(Landing Pad) → land(Landing Pad)
Validation	Passed
Execution	Mission completed successfully

4 Real-life Challenges of Long-term Robot Deployments

4.1 Sensor Degradation

Long-term robotic deployments expose sensors to dust, vibration, temperature fluctuations, moisture, and mechanical wear. Over time, camera lenses accumulate dirt, LiDAR calibration may drift, and inertial sensors develop increasing bias. These effects gradually reduce perception accuracy and localization performance. Robust robotic systems therefore require sensor health monitoring, periodic recalibration, and fault detection mechanisms to maintain reliable operation throughout extended deployments [11].

4.2 Communication Constraints

Robots operating in large-scale environments frequently encounter unreliable communication networks. Warehouses, underground mines, industrial facilities, and disaster zones often experience packet loss, latency, or complete communication outages. Consequently, robots must be capable of autonomous operation even when disconnected from a central controller. Effective long-term deployments require distributed decision-making, local data storage, and graceful degradation strategies that allow mission execution despite intermittent connectivity [1].

4.3 Energy Constraints

Energy availability remains one of the primary limitations of long-duration robotic missions. Battery capacity degrades over repeated charge-discharge cycles, reducing operational endurance and increasing maintenance requirements. For aerial robots, energy constraints directly limit flight duration and mission coverage. Future autonomous systems must incorporate energy-aware planning, adaptive task allocation, and intelligent charging strategies to maximize operational efficiency while ensuring safe mission completion under limited power resources.

4.4 Foundation Models Cannot Have Full Authority

Foundation models have demonstrated strong capabilities in reasoning, planning, and natural language understanding. However, their outputs may contain hallucinations, incorrect assumptions, or unsafe actions when deployed in unfamiliar situations. For safety-critical robotic applications, foundation models should be restricted to high-level planning and decision support. Deterministic validation, rule-based safety constraints, collision avoidance systems, and human oversight must remain responsible for enforcing operational safety and preventing hazardous behavior [8].

4.5 Environment Changes Over Time

Real-world environments continuously evolve, making static maps progressively inaccurate. Furniture may be relocated, vehicles change position, construction activities alter layouts, and seasonal effects modify visual appearance. These semantic and geometric changes can significantly degrade localization and navigation performance over time. Long-term autonomy therefore requires continual map updates, adaptive world models, and semantic understanding that allows robots to recognize and respond to environmental changes without requiring complete remapping [11].

5 Highlighting an Open Research Problem

5.1 Current Challenges in Long-Term Inspection Autonomy

Recent advances in robotic inspection have enabled autonomous systems to operate in industrial facilities, warehouses, construction sites, and other large-scale environments for extended periods. Improvements in simultaneous localization and mapping (SLAM), semantic perception, and mission planning

have significantly increased the autonomy of robotic platforms. However, long-term inspection remains challenging because most existing systems assume relatively static environments and reliable sensing conditions. In practice, environmental appearance changes over time due to moving objects, seasonal variations, structural modifications, lighting changes, and sensor degradation. These factors reduce localization accuracy and can lead to incorrect environmental understanding. Furthermore, many inspection systems still rely on periodic human intervention for map maintenance, task validation, and recovery from unexpected failures, limiting their ability to achieve truly persistent autonomy [11].

5.2 Impact of Real-World Conditions and Multi-Robot Operation

Real-world deployments frequently violate assumptions commonly made in research literature. Sensors experience noise, calibration drift, occlusions, adverse weather conditions, and communication interruptions. In large-scale environments, robots may encounter regions that differ significantly from the data used during training, causing perception and decision-making models to perform unpredictably. When multiple robots operate simultaneously, additional challenges emerge. Shared maps must remain consistent across all robots despite asynchronous observations and communication delays. Task allocation must avoid duplication of effort while ensuring balanced workload distribution. Furthermore, failures of individual robots can propagate through the system if remaining robots are unable to dynamically redistribute unfinished tasks. As the number of robots increases, maintaining accurate semantic world models and coordinated decision-making becomes increasingly difficult.

5.3 Proposed Research Direction

A promising research direction is the development of a distributed semantic world model that continuously evolves through observations collected by multiple robots. Instead of relying on a static map, each robot would contribute updates to a shared semantic scene graph representing both geometric and semantic information about the environment. Foundation models could be used for high-level reasoning, task decomposition, and interpretation of semantic changes, while deterministic validation layers would ensure safety and correctness. The key research challenge is enabling robots to autonomously determine which environmental changes are temporary, which are permanent, and how those changes should influence future inspection missions. Such a system could improve long-term autonomy by allowing robot teams to adapt their world models, mission plans, and task allocations without requiring continuous human supervision. This capability would represent an important step toward persistent autonomous inspection in dynamic real-world environments.

6 Time Spent and AI Usage Declaration

Approximate Time Spent

Task	Time Spent
Gazebo Environment and Simulation Setup	4 hours
Mission Planning and Task Allocation	3 hours
Failure Recovery Implementation	1 hour
Semantic Planning Framework	2 hours
Testing and Validation	3 hours
Report Writing and Documentation	3 hours
Total	16 hours

Use of AI Tools

AI-assisted tools were used for report writing, proofreading, literature discovery, and generation of diagrams and LaTeX formatting.

Verification was performed through simulation-based testing in Gazebo and ArduPilot, inspection of generated outputs, debugging of implementation errors, cross-checking of mathematical derivations, and manual proofreading of all report sections. No AI-generated content was included without verification and modification where necessary.

References

- [1] A. Khamis, A. Hussein, and A. Elmogy, “Multi-Robot Task Allocation: A Review of the State-of-the-Art,” *Cooperative Robots and Sensor Networks*, 2015.
- [2] B. Kartal, E. Nunes, J. Godoy, and M. Gini, “Monte Carlo Tree Search for Multi-Robot Task Allocation,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2016.
- [3] J. E. Normey-Rico and E. F. Camacho, “Mobile Robot Path Tracking Using a Robust PID Controller,” *Control Engineering Practice*, vol. 9, no. 11, pp. 1209–1214, 2001.
- [4] W. Cai, Y. Zhang, and X. Wang, “Quadrotor Waypoint-Tracking Control Under Exogenous Disturbances,” *Journal of the Franklin Institute*, vol. 357, no. 12, pp. 7702–7726, 2020.
- [5] S. Z. U. Abideen, M. A. Khan, and others, “Multi-Robot Task Assignment: A Survey of Algorithmic Approaches, Challenges, and Future Directions,” *Computer Science Review*, vol. 50, 2026.
- [6] M. Elango, S. Nachiappan, and M. K. Tiwari, “Balancing Task Allocation in Multi-Robot Systems Using K-Means Clustering and Auction Based Mechanisms,” *Expert Systems with Applications*, vol. 38, no. 6, pp. 6486–6491, 2011.
- [7] J. Camacho, R. Rivas-Perea, and others, “A Waypoint-Based Guidance Algorithm for Mini UAVs,” *IFAC Proceedings Volumes*, vol. 46, no. 30, pp. 120–125, 2013.
- [8] W. Huang, F. Xia, T. Xiao, A. Chan, J. Liang, J. Florensa, A. Zeng, J. Thompson, and F. B. Li, “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents,” *ICML*, 2022.
- [9] I. Singh, V. Blukis, A. Mousavian, F. Xia, D. Zeng, J. Thompson, A. Gupta, P. Abbeel, and A. Zeng, “ProgPrompt: Generating Situated Robot Task Plans using Large Language Models,” *ICRA*, 2023.
- [10] L. P. Kaelbling and T. Lozano-Perez, “Hierarchical Task and Motion Planning in the Now,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1470–1477, 2011.
- [11] P. Biber and T. Duckett, “Dynamic Maps for Long-Term Operation of Mobile Service Robots,” In *Proceedings of Robotics: Science and Systems (RSS)*, Cambridge, Massachusetts, USA, pp. 17–24, 2005.